UNIVERSIDAD DON BOSCO	**UNIVERSIDAD DON BOSCO** **FACULTAD DE INGENIERIA** **ESCUELA DE COMPUTACIÓN**
CICLO II	**GUIA DE LABORATORIO #5** **Programación Orientada a Objetos** **Creación de interfaces graficas con SWING**

I.OBJETIVOS

Que el estudiante:

- Cree interfaces gráficas con SWING utilizando las herramientas de IntellJ IDEA.
- Maneje a diferentes eventos de los componentes graficos, según sea la necesidad.

II. INTRODUCCIÓN

Java posee un juego de clases capaces de construir potentes interfaces gráficas de usuarios (**GUI: Graphical User Interface**). A este grupo de clases se les denomina JFC Java Fundation Classes y presentan las siguientes características:

- Diseño de interfaces puramente Java y por lo tanto independientemente de la plataforma donde se ejecute.
- Extensible y personalizable: se trata de un framework intuitivo, ampliable por el programador, extendiendo e implementando las clases base. Es totalmente personalizable a nivel de look and feel, siendo posible aplicar diferentes tipos de apariencias de las ventanas, en principio, independientemente del sistema operativo sobre el que se ejecute.

La interfaz gráfica de java posee los siguientes elementos comunes:

- Contenedores: se pueden distinguir dos tipos: los contenedores base, que representan al marco de la ventana y son necesarios al menos uno por aplicación, y los contenedores que agrupan a los componentes, considerando este último tipo de contenedor como un componente que agrupa otros componentes bajo un mismo panel.
- Componentes: controles que se insertarán en los contendedores (botones, cajas de texto…).
- Layouts: representan la organización de los controles sobre un contenedor.
- Gestión de eventos: Mecanismo por el cual es posible capturar y procesar los eventos del usuario producidos sobre el contenedor y sus controles.

¿Qué es Swing?

Swing es una biblioteca de paquetes con clases que permite crear interfaces gráficas de usuario en Java y su apariencia de visualización no depende del entorno (Sistema operativo).

Swing forma parte del paquete estándar, no hace falta importar ningún fichero adicional en el proyecto que sea externo de la API que ya tiene el lenguaje. No se trata de un nuevo framework de desarrollo en sistemas de ventanas, si no que más bien se basa en AWT. Su aparición se dio en la versión 1.2 de Java.

La librería está implementada en 18 paquetes, de los cuales, los más utilizados son los siguientes:

- javax.swing: es el paquete de más alto nivel, que contiene los componentes, adaptadores, los modelos por defecto de los componentes, y las interfaces para todos los modelos.
- javax.swing.events: contiene los tipos de eventos y listeners (oyentes) especificos de Swing. Ademas, los componentes Swing pueden generar sus propios eventos.

Las clases de esta librería permiten diseñar ventanas, las cuales pueden incluir botones, cajas de texto, archivos multimedia (imágenes, audio, video), etc. Y finalmente, mostrar estas ventanas al ejecutar la aplicación de java.

La primer clase en importancia de Swing es **JFrame**. Esta clase es una ventana que tiene un contenedor en el que podemos poner controles.

JFrame es una ventana normal de Windows. Dentro de un JFrame existe algo llamado ContentPane. Un **ContentPane** es un contenedor de controles en el que podemos agregar los elementos de la interfaz gráfica.

Entre las clases para presentar controles en un ContentPane se tiene a **JLabel**, que genera etiquetas para mostrar texto en nuestras aplicaciones. Permite agregar texto, cambiar el formato, la posición, agregar imágenes y muchas cosas más.



III. PROCEDIMIENTO

En el mundo profesional de Java, las interfaces gráficas se programan a través de código y no arrastrando elementos. Esto garantiza que nuestras ventanas sean **responsivas** mediante el uso de "Gestores de Diseño" (Layout Managers) y que el código sea limpio y mantenible aplicando la arquitectura **MVC (Modelo - Vista - Controlador)**.

Para ello, crearemos un proyecto y lo dividiremos en dos pequeñas aplicaciones: Un **Gestor de Personas** y un **Visor de Imágenes**.

Paso 1: Configuración del Proyecto y Paquetes

1. Abra **IntelliJ IDEA** y cree un nuevo proyecto llamado *POOGuia5*.
2. Copie la carpeta de imágenes proporcionada para esta práctica (img) y péguela dentro de su carpeta src.

3. Dentro de src, cree la siguiente estructura de paquetes:
    - sv.edu.udb.vistas
    - sv.edu.udb.controladores

Paso 2: La Vista (Diseño del Formulario y Tabla)

Navegue a sv.edu.udb.vistas y cree la clase *VistaPersona*. Aquí utilizaremos GridLayout para el formulario y BorderLayout para acomodar los componentes de la siguiente tabla:

Control	Propiedad	Valor
JLabel1	text	Ingrese su id:
	field name	lblId
JLabel2	text	Ingrese su nombre:
	field name	lblNombre
JLabel3	text	Ingrese su edad:
	field name	lblEdad
JLabel4	text	Ingrese su teléfono:
	field name	lblTelefono
JLabel5	text	Seleccione sexo:
	field name	lblSexo
JTextField1	field name	txtId
JTextField2	field name	txtNombre
JTextField3	field name	txtEdad
JTextField4	field name	txtTelefono
JComboBox1	field name	cmbSexo
	model	Masculino Femenino
JButton1	text	Obtener Datos
	field name	btnObtenerDatos
JButton2	text	Limpiar
	field name	btnLimpiar

El código para esta clase sería el siguiente:

```
package sv.edu.udb.vistas;

import javax.swing.*;
import javax.swing.table.DefaultTableModel;
import java.awt.*;
```

```
// La clase hereda de JFrame para convertirse en el contenedor principal
public class VistaPersona extends JFrame {

    // Controles del formulario
    public JTextField txtId, txtNombre, txtEdad, txtTelefono;
    public JComboBox<String> cmbSexo;
    public JButton btnObtenerDatos, btnLimpiar;

    // Controles de la Tabla
    public JTable tblDatos;
    public DefaultTableModel modeloTabla;

    public VistaPersona() {
        setTitle("Gestión de Personas - MVC");
        setSize(600, 500);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null); // Centra la ventana
        setLayout(new BorderLayout(10, 10)); // Gestor de diseño cardinal

        inicializarFormulario();
        inicializarTabla();
    }

    private void inicializarFormulario() {
        // Cuadrícula de 6 filas por 2 columnas
        JPanel pnlFormulario = new JPanel(new GridLayout(6, 2, 5, 5));
        pnlFormulario.setBorder(BorderFactory.createTitledBorder("Datos de
Persona"));

        pnlFormulario.add(new JLabel("Ingrese su ID:"));
        txtId = new JTextField();
        pnlFormulario.add(txtId);

        pnlFormulario.add(new JLabel("Ingrese su nombre:"));
        txtNombre = new JTextField();
        pnlFormulario.add(txtNombre);

        pnlFormulario.add(new JLabel("Ingrese su edad:"));
        txtEdad = new JTextField();
        pnlFormulario.add(txtEdad);

        pnlFormulario.add(new JLabel("Ingrese su teléfono:"));
        txtTelefono = new JTextField();
        pnlFormulario.add(txtTelefono);

        pnlFormulario.add(new JLabel("Seleccione sexo:"));
        cmbSexo = new JComboBox<>(new String[]{"Masculino", "Femenino"});
        pnlFormulario.add(cmbSexo);

        btnObtenerDatos = new JButton("Obtener Datos");
        btnLimpiar = new JButton("Limpiar");
        pnlFormulario.add(btnObtenerDatos);
        pnlFormulario.add(btnLimpiar);

        add(pnlFormulario, BorderLayout.NORTH);
    }
```

```
    private void inicializarTabla() {
        // El Modelo (TableModel) es la estructura que alimenta a la JTable
        String[] columnas = {"ID", "Nombres", "Edad", "Teléfono", "Sexo"};
        modeloTabla = new DefaultTableModel(null, columnas);

        tblDatos = new JTable(modeloTabla);
        // JScrollPane agrega las barras de desplazamiento[cite: 7]
        JScrollPane scrollPane = new JScrollPane(tblDatos);

        add(scrollPane, BorderLayout.CENTER);
    }
}
```

Paso 3: El Controlador (Gestión de Eventos)

Navegue a sv.edu.udb.controladores y cree la clase *PersonaController*. Usaremos Expresiones Lambda para capturar los eventos (ActionListener y MouseListener):

```
package sv.edu.udb.controladores;

import sv.edu.udb.vistas.VistaPersona;
import javax.swing.JOptionPane;
import java.awt.event.MouseAdapter;
import java.awt.event.MouseEvent;

public class PersonaController {
    private VistaPersona vista;

    public PersonaController(VistaPersona vista) {
        this.vista = vista;
        inicializarEventos();
    }

    private void inicializarEventos() {
        // Expresiones Lambda para simplificar los eventos de los botones
        vista.btnLimpiar.addActionListener(e -> limpiarControles());
        vista.btnObtenerDatos.addActionListener(e -> capturarDatos());

        // Evento para leer datos al hacer clic en una fila de la tabla
        vista.tblDatos.addMouseListener(new MouseAdapter() {
            @Override
            public void mouseClicked(MouseEvent e) {
                obtenerFilaDeTabla();
            }
        });
    }

    private void limpiarControles() {
        vista.txtId.setText("");
        vista.txtNombre.setText("");
        vista.txtEdad.setText("");
        vista.txtTelefono.setText("");
        vista.cmbSexo.setSelectedIndex(0);
        vista.txtId.requestFocus();
    }
```

```
    private void capturarDatos() {
        if(vista.txtId.getText().isEmpty() || vista.txtNombre.getText().isEmpty())
{
            JOptionPane.showMessageDialog(vista, "Ingrese al menos el ID y el
Nombre.");
            return;
        }

        // Agregamos los datos al Modelo de la Tabla como una nueva Fila
        Object[] nuevaFila = {
                vista.txtId.getText(),
                vista.txtNombre.getText(),
                vista.txtEdad.getText(),
                vista.txtTelefono.getText(),
                vista.cmbSexo.getSelectedItem().toString()
        };
        vista.modeloTabla.addRow(nuevaFila);
        limpiarControles();
    }

    private void obtenerFilaDeTabla() {
        int fila = vista.tblDatos.getSelectedRow();
        if (fila > -1) {
            // Extraemos los valores de la tabla y los ponemos en las cajas de
texto
            vista.txtId.setText(vista.modeloTabla.getValueAt(fila, 0).toString());
            vista.txtNombre.setText(vista.modeloTabla.getValueAt(fila,
1).toString());
            vista.txtEdad.setText(vista.modeloTabla.getValueAt(fila,
2).toString());
            vista.txtTelefono.setText(vista.modeloTabla.getValueAt(fila,
3).toString());
            vista.cmbSexo.setSelectedItem(vista.modeloTabla.getValueAt(fila,
4).toString());
        }
    }
}
```

Paso 4: La Vista del Selector de Imágenes

En el paquete sv.edu.udb.vistas, cree la clase *VistaImagenes*:

Control	Propiedad	Valor
JPanel	**field name**	pnlImg
JLabel1	**text**	Seleccione una imagen
	field name	lblTitulo
	Horizontal Align	Center
	font	Segoe Print, Bold, 22 pt
JLabel2	**text**	(vacio)
	icon	sv/edu/udb/img/img1.jpeg

	field name	lblImagen
JRadioButton1	**text**	Opcion 1
	field name	rbtOpcion1
	selected	(marcado)
	buttonGroup	buttonGroup1
JRadioButton2	**text**	Opcion 2
	field name	rbtOpcion2
	buttonGroup	buttonGroup1
JRadioButton3	**text**	Opcion 3
	field name	rbtOpcion3
	buttonGroup	buttonGroup1

```
package sv.edu.udb.vistas;

import javax.swing.*;
import java.awt.*;

public class VistaImagenes extends JFrame {
    public JRadioButton rbtOpcion1, rbtOpcion2, rbtOpcion3;
    public JLabel lblImagen;

    public VistaImagenes() {
        setTitle("Selección de Imágenes");
        setSize(400, 300);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
        setLayout(new BorderLayout(10, 10));

        JLabel lblTitulo = new JLabel("Seleccione una imagen",
SwingConstants.CENTER);
        lblTitulo.setFont(new Font("Segoe Print", Font.BOLD, 22)); //
Personalización de fuente
        add(lblTitulo, BorderLayout.NORTH);

        // Panel lateral izquierdo para los RadioButtons
        JPanel pnlOpciones = new JPanel(new GridLayout(3, 1));
        rbtOpcion1 = new JRadioButton("Opción 1", true); // Seleccionado por
defecto
        rbtOpcion2 = new JRadioButton("Opción 2");
        rbtOpcion3 = new JRadioButton("Opción 3");

        // ButtonGroup agrupa los controles para que sean mutuamente excluyentes
        ButtonGroup grupoOpciones = new ButtonGroup();
        grupoOpciones.add(rbtOpcion1);
        grupoOpciones.add(rbtOpcion2);
        grupoOpciones.add(rbtOpcion3);

        pnlOpciones.add(rbtOpcion1);
        pnlOpciones.add(rbtOpcion2);
```

```
        pnlOpciones.add(rbtOpcion3);
        add(pnlOpciones, BorderLayout.WEST);

        // Etiqueta central que contendrá la imagen[cite: 7]
        lblImagen = new JLabel("", SwingConstants.CENTER);
        add(lblImagen, BorderLayout.CENTER);
    }
}
```

Paso 5: El Controlador de Imágenes

En el paquete sv.edu.udb.controladores, cree la clase *ImagenesController*. Usaremos la clase ImageIcon para cargar los archivos del proyecto:

```
package sv.edu.udb.controladores;

import sv.edu.udb.vistas.VistaImagenes;
import javax.swing.ImageIcon;
import java.net.URL;

public class ImagenesController {
    private VistaImagenes vista;

    public ImagenesController(VistaImagenes vista) {
        this.vista = vista;
        inicializarEventos();
        cambiarImagen("/img/img1.jpeg"); // Carga la primera imagen por defecto
    }

    private void inicializarEventos() {
        // Al seleccionar un radio button, se cambia la imagen dinámicamente
        vista.rbtOpcion1.addActionListener(e -> cambiarImagen("/img/img1.jpeg"));
        vista.rbtOpcion2.addActionListener(e -> cambiarImagen("/img/img2.jpeg"));
        vista.rbtOpcion3.addActionListener(e -> cambiarImagen("/img/img3.jpeg"));
    }

    private void cambiarImagen(String ruta) {
        URL urlImagen = getClass().getResource(ruta);
        if (urlImagen != null) {
            vista.lblImagen.setIcon(new ImageIcon(urlImagen)); // Renderiza el
objeto ImageIcon
        } else {
            vista.lblImagen.setText("Imagen no encontrada");
            vista.lblImagen.setIcon(null);
        }
    }
}
```

Paso 6: Orquestador Principal

Para probar sus aplicaciones, cree la clase *Aplicacion* en el paquete raíz sv.edu.udb. Utilice SwingUtilities para garantizar que la interfaz se ejecute en un hilo de memoria seguro:

```java
package sv.edu.udb;

import sv.edu.udb.controladores.PersonaController;
import sv.edu.udb.vistas.VistaPersona;
import sv.edu.udb.controladores.ImagenesController;
import sv.edu.udb.vistas.VistaImagenes;
import javax.swing.SwingUtilities;

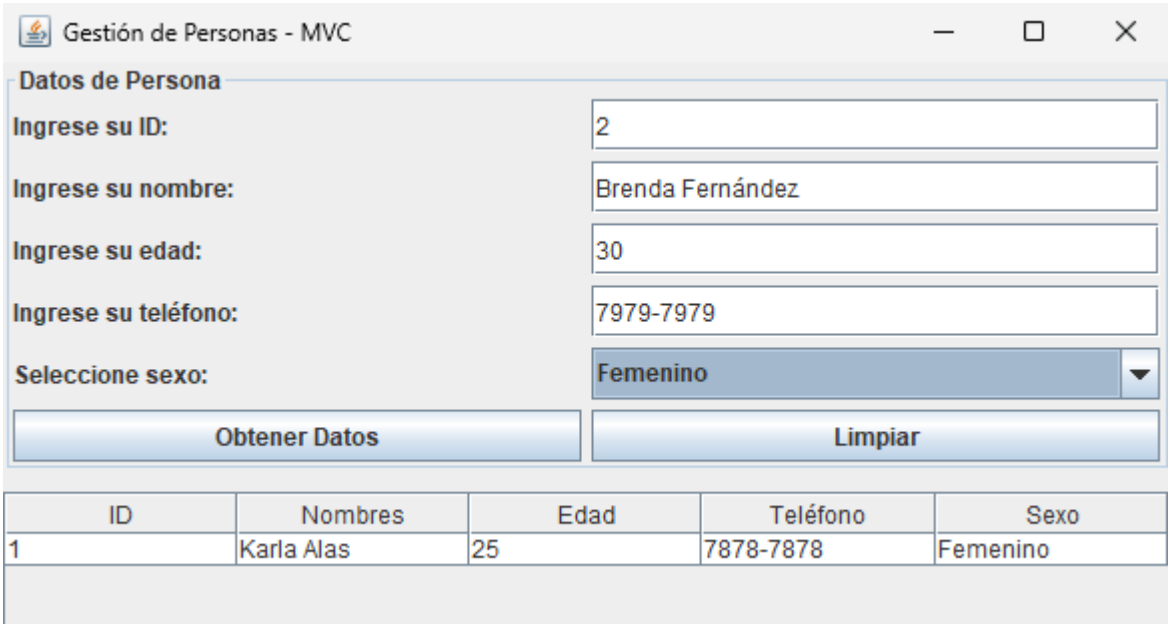
public class Aplicacion {
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {

            // --- PARA PROBAR PRIMERA PARTE (Formulario y Tabla) ---
            VistaPersona vistaPersona = new VistaPersona();
            new PersonaController(vistaPersona);
            vistaPersona.setVisible(true);

            // --- PARA PROBAR SEGUNDA PARTE (Visor de Imágenes) ---
            // Descomente las siguientes 3 líneas y comente las de arriba para
probarlo:
            // VistaImagenes vistaImg = new VistaImagenes();
            // new ImagenesController(vistaImg);
            // vistaImg.setVisible(true);

        });
    }
}
```

La ejecución de la primera parte generaría una salida como la siguiente:



Realice diferentes pruebas, mientras que la segunda parte, obtendremos una ventana como la siguiente:



IV. DISCUSION DE RESULTADOS

Ejercicio

- Investigue que es un archivo CSV y como se procesan utilizando Java.
- Luego, modifique al JFrame de la clase Persona (realizado en la Parte III del Procedimiento de esta practica) para que al iniciar su ejecucion, recupere de un archivo CSV a los registros de un conjunto de Personas y los muestre en el control tblDatos existente.
- Por último, haga los cambios para que al capturar los datos de una nueva persona, esta información también se añada como un registro mas dentro del archivo CSV usado al iniciar la aplicación.

V. BIBLIOGRAFIA

- Ruíz, Isaac. *JTable con Ejemplos-Parte I* [Archivo PDF]. Recuperado de http://www.javahispano.org/contenidos/archivo/63/jtable_1.pdf.

- ORACLE Java Documentation (2022), *The Java TM Tutorials, How to Use Tables*. https://docs.oracle.com/javase/tutorial/uiswing/components/table.html